

E-commerce Website Search Automation

Software Testing

Samadhi Hansika – UGC_001/B005-A/0005

Dhakshina Deshan – UGC_001/B005-B/0009

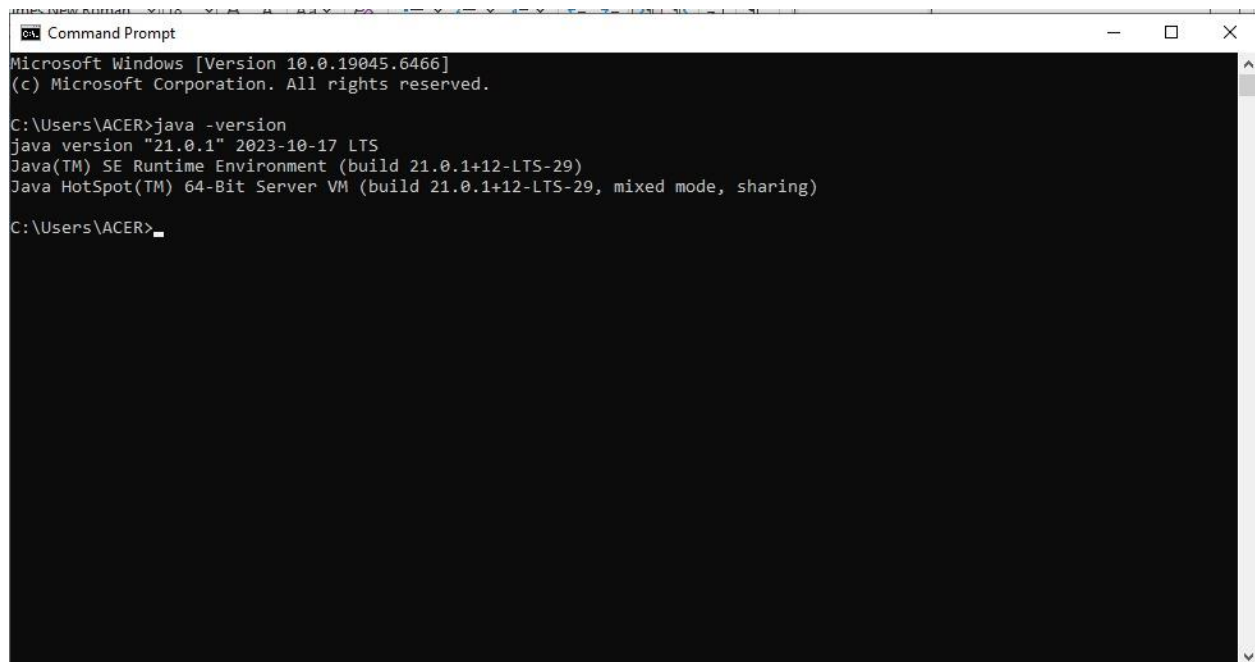
Nurani Kaushalya – UGC_001/B005-B/0002

Introduction

The objective of this assignment is to demonstrate the automation of a "Search" functionality on an e-commerce website using Selenium WebDriver. The script is designed to search for the term "Laptop" and verify that the results are displayed correctly. The project is implemented using Java and managed via Maven.

Step 1: Environment Setup

Java Version Verification



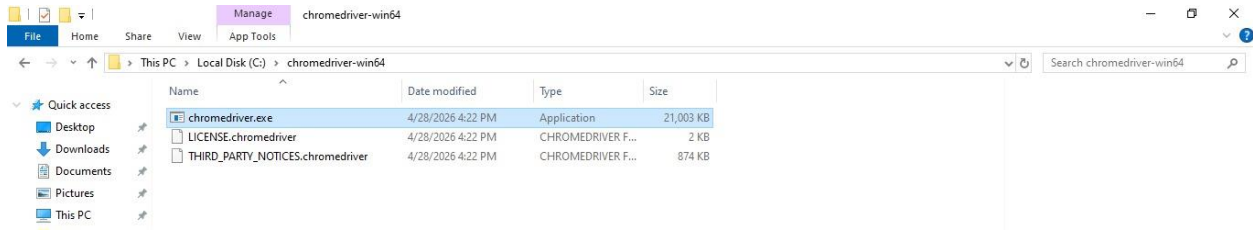
```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>java -version
java version "21.0.1" 2023-10-17 LTS
Java(TM) SE Runtime Environment (build 21.0.1+12-LTS-29)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.1+12-LTS-29, mixed mode, sharing)

C:\Users\ACER>
```

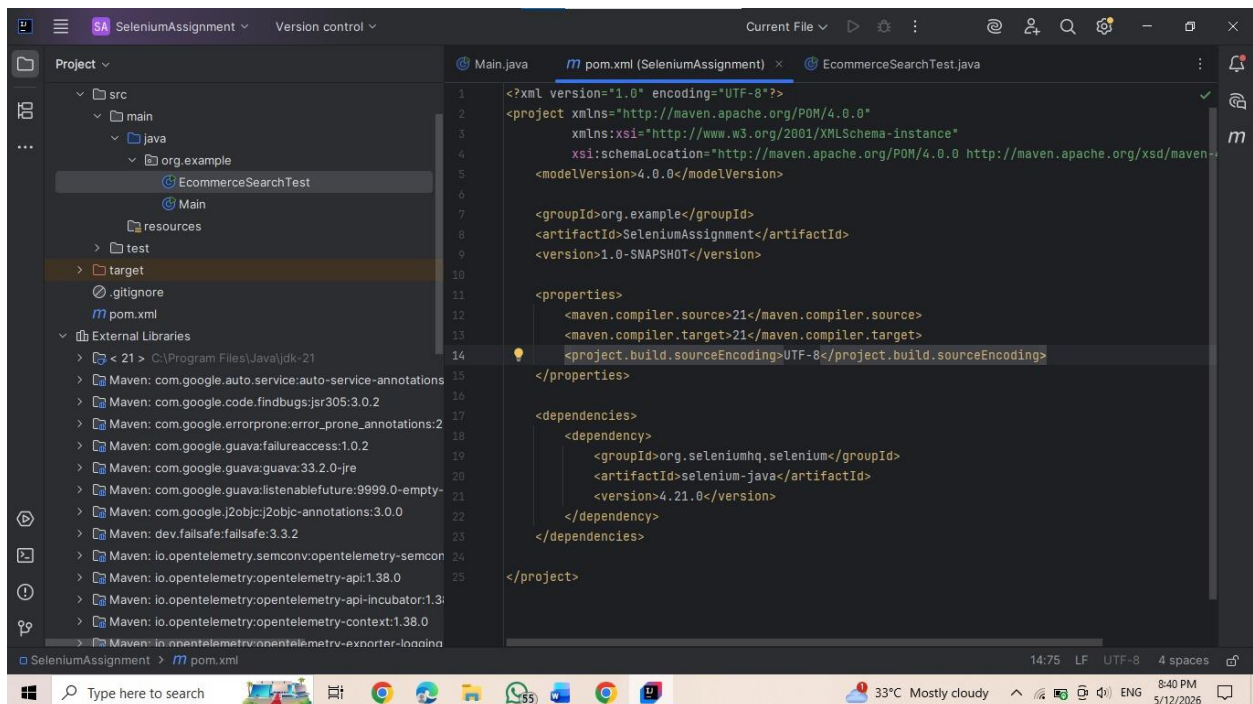
Verification of the Java Development Kit (JDK) installation is confirmed via the terminal. Executing the `java -version` command ensures that the system environment variables are correctly configured, allowing the IDE to compile and run Java-based automation scripts.

ChromeDriver Executable



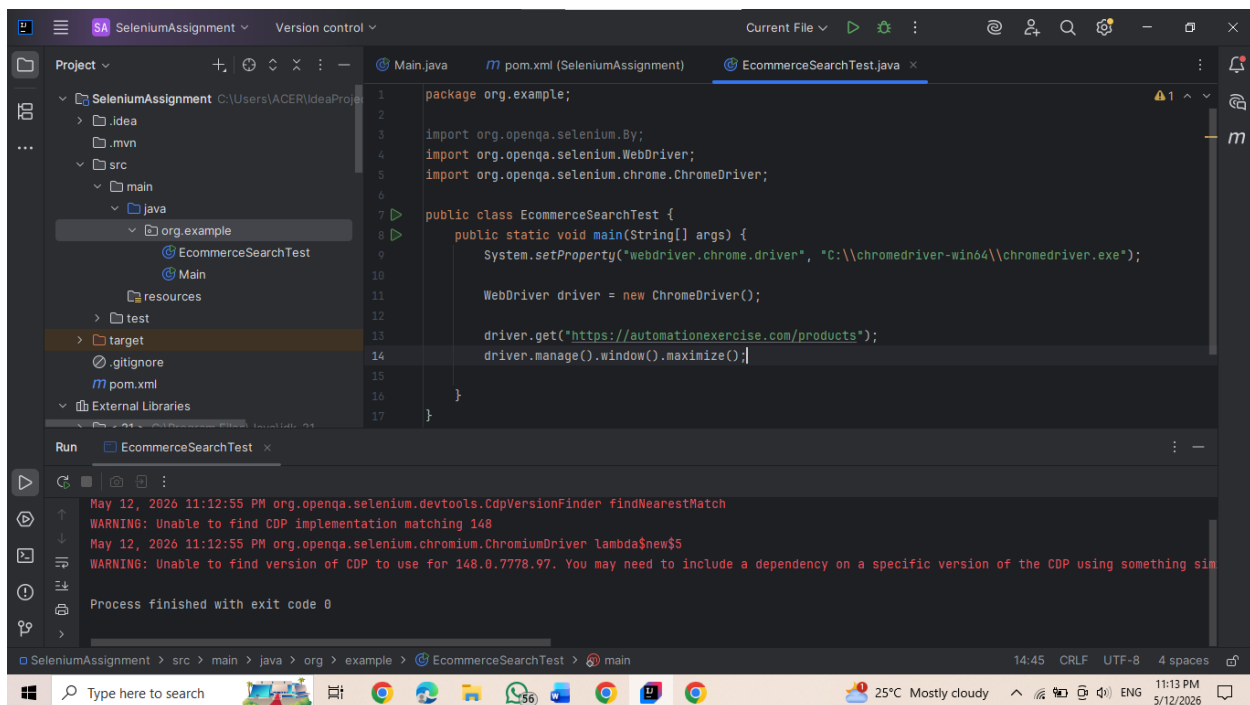
The ChromeDriver executable is stored in a local directory, ensuring that the driver version is compatible with the installed version of the Google Chrome browser. This file acts as the essential bridge between the Selenium WebDriver commands and the browser engine.

Step 2: Project Configuration & Libraries



The project's pom.xml file within the IntelliJ IDEA environment serves as the central configuration for dependency management. The selenium-java dependency is integrated into the Maven build script to facilitate the automatic download of necessary libraries. This ensures that all classes required for browser interaction are available within the project classpath.

Step 3: Initializing the Browser



The screenshot displays the IntelliJ IDEA IDE interface. The left sidebar shows the project structure for 'SeleniumAssignment', with the 'EcommerceSearchTest' class selected under the 'org.example' package. The main editor window shows the code for 'EcommerceSearchTest.java', which includes imports for Selenium WebDriver and ChromeDriver, and a main method that initializes the WebDriver and navigates to a specific URL. The bottom panel shows the Run console output, which includes warnings about CDP implementation and version, and a final message indicating the process finished with exit code 0.

```
package org.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

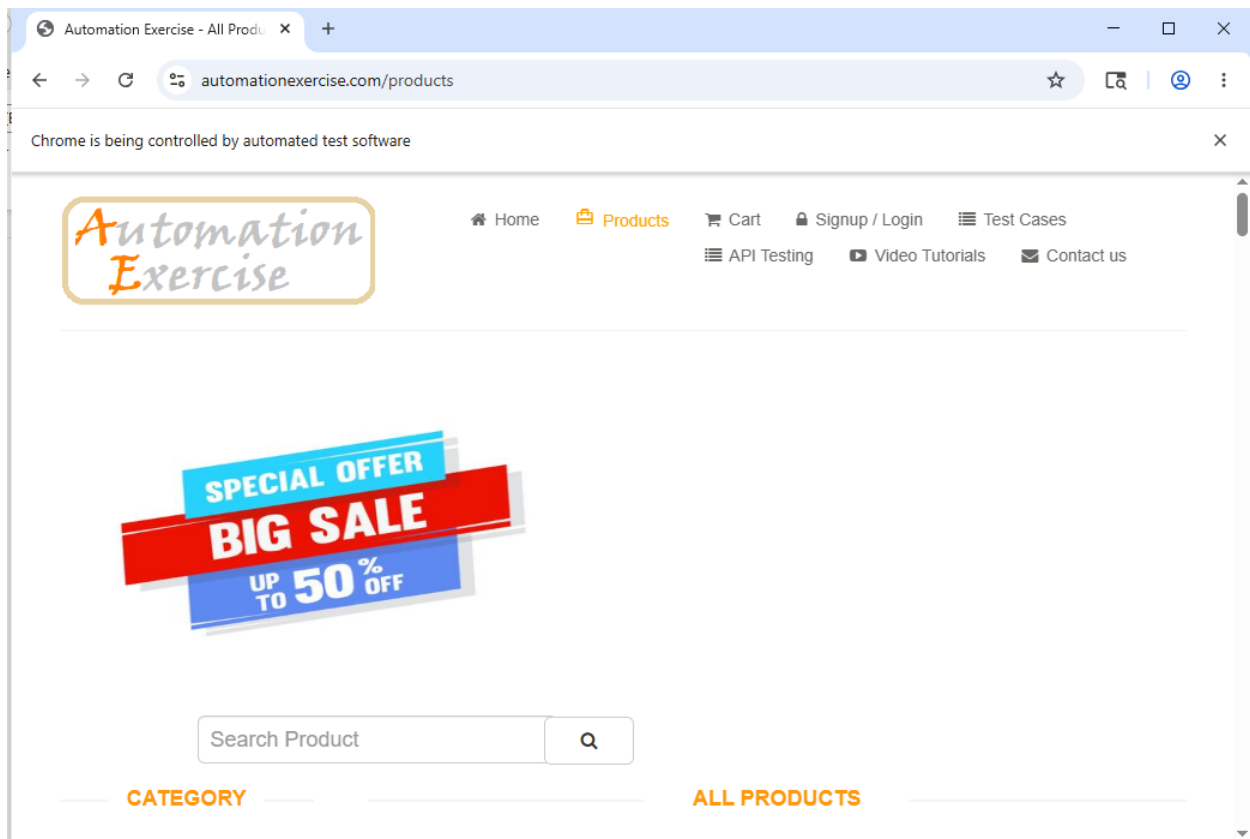
public class EcommerceSearchTest {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "C:\\\\chromedriver-win64\\\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        driver.get("https://automationexercise.com/products");
        driver.manage().window().maximize();
    }
}
```

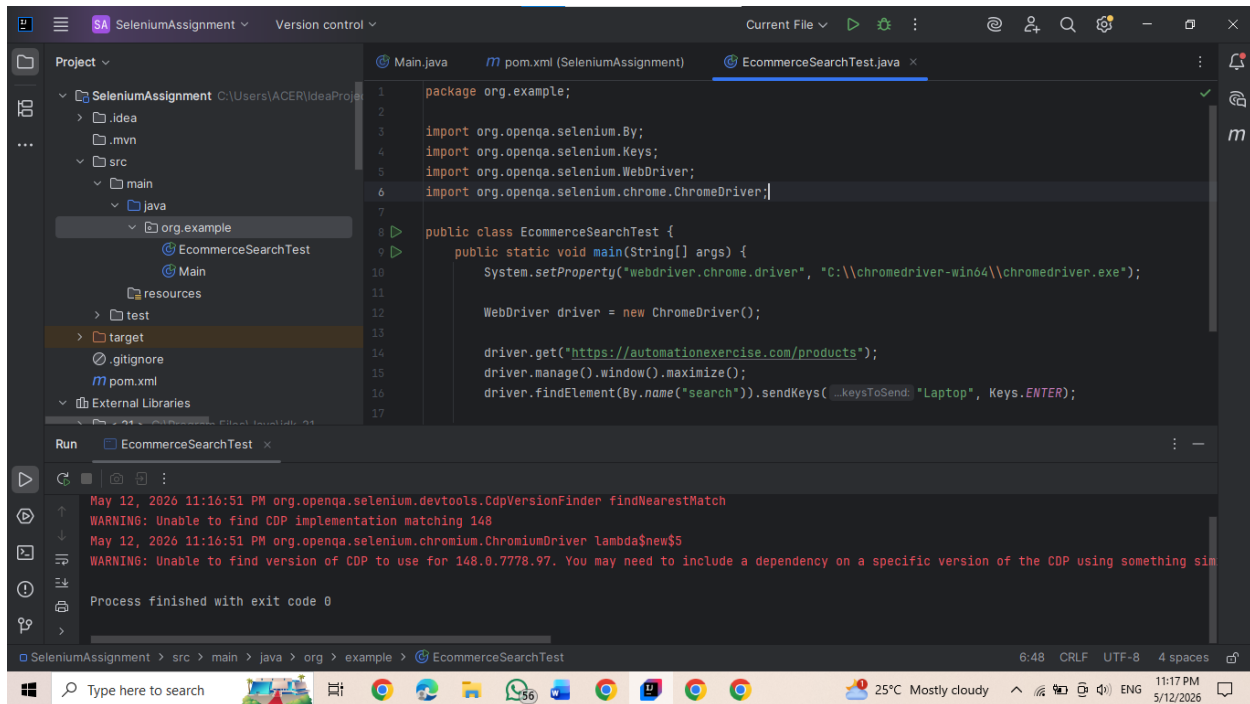
Run console output:

```
May 12, 2026 11:12:55 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 148
May 12, 2026 11:12:55 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$5
WARNING: Unable to find version of CDP to use for 148.0.7778.97. You may need to include a dependency on a specific version of the CDP using something similar
Process finished with exit code 0
```



The core implementation of the automation script is established within the IntelliJ IDEA environment using Java. The `System.setProperty` method is utilized to define the driver's executable path, linking the script to the local `chromedriver.exe` directory. By instantiating the `ChromeDriver` class, the script triggers a new browser instance and uses the `driver.get()` method to navigate to the OpenCart demo website. The console output verifies a successful execution sequence, finishing with 'exit code 0,' which confirms that the environment setup and initial navigation were completed without fatal errors.

Step 4: The Automation Logic



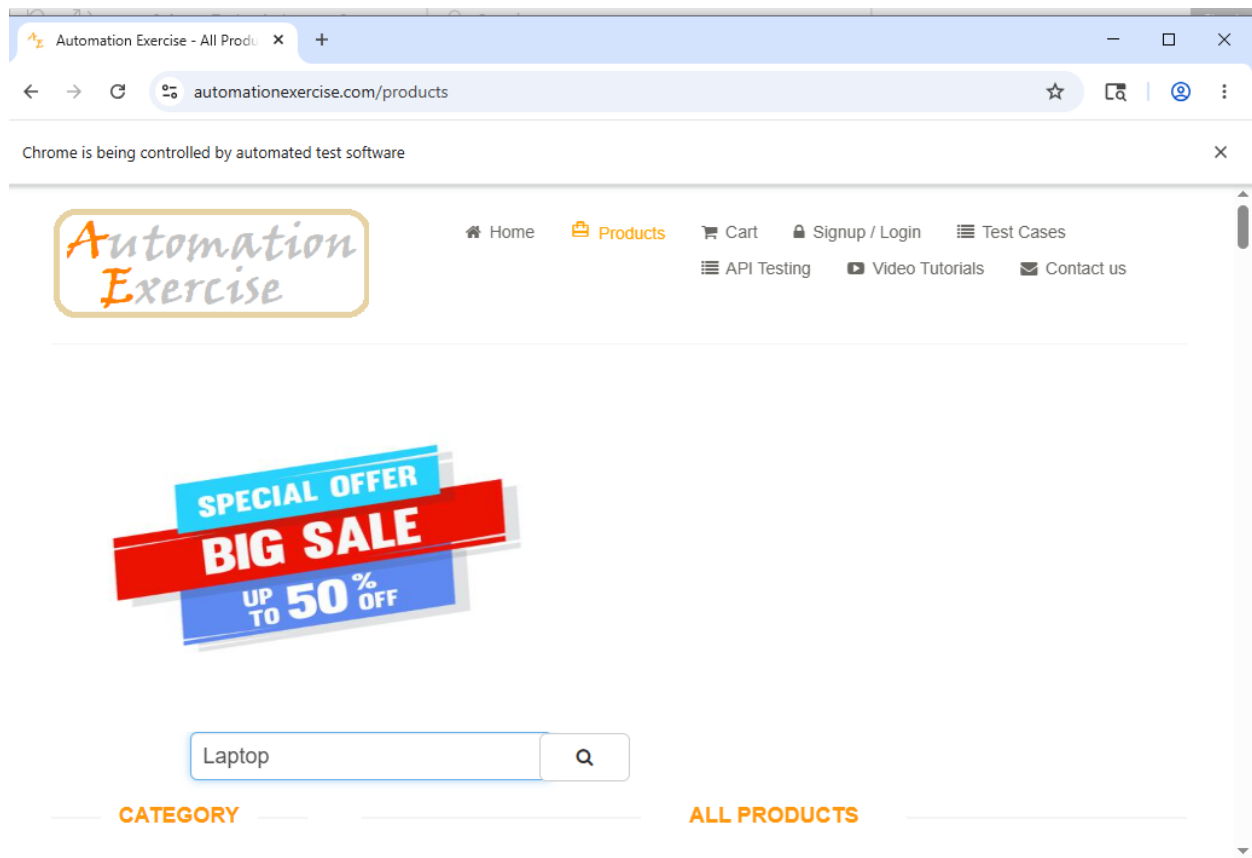
The screenshot displays an IDE window for a project named "SeleniumAssignment". The left sidebar shows the project structure with folders like .idea, .mvn, src, main, java, org.example, and test. The main editor area shows the code for "EcommerceSearchTest.java". The code defines a package, imports Selenium and ChromeDriver, and implements a main method that sets the WebDriver path, creates a ChromeDriver instance, navigates to a website, maximizes the window, and sends keys to a search field.

```
1 package org.example;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.Keys;
5 import org.openqa.selenium.WebDriver;
6 import org.openqa.selenium.chrome.ChromeDriver;
7
8 public class EcommerceSearchTest {
9     public static void main(String[] args) {
10         System.setProperty("webdriver.chrome.driver", "C:\\\\chromedriver-win64\\\\chromedriver.exe");
11
12         WebDriver driver = new ChromeDriver();
13
14         driver.get("https://automationexercise.com/products");
15         driver.manage().window().maximize();
16         driver.findElement(By.name("search")).sendKeys(KeysToSend.LAPTOP, Keys.ENTER);
17     }
18 }
```

The bottom panel shows the output of the test run. It includes timestamps, log messages, and a warning about the CDP version. The process finished with exit code 0.

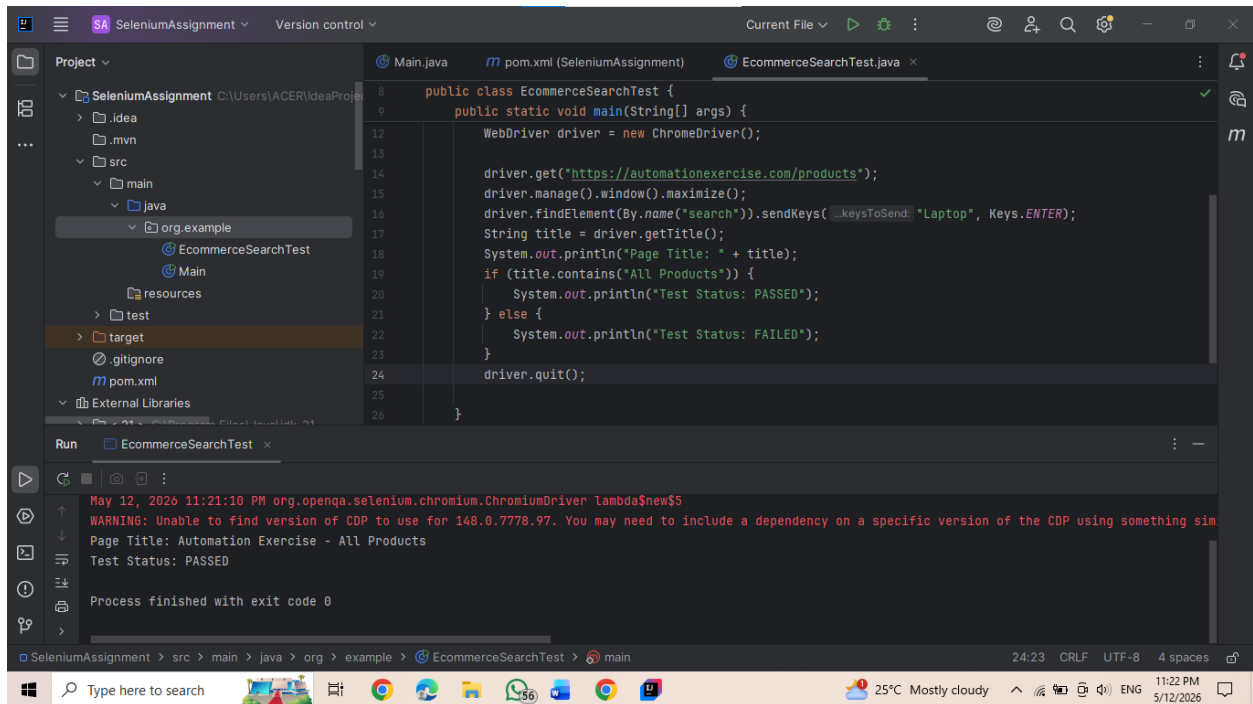
```
May 12, 2026 11:16:51 PM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find CDP implementation matching 148
May 12, 2026 11:16:51 PM org.openqa.selenium.chromium.ChromiumDriver lambda$new$5
WARNING: Unable to find version of CDP to use for 148.0.7778.97. You may need to include a dependency on a specific version of the CDP using something similar
Process finished with exit code 0
```

The status bar at the bottom indicates the file encoding is UTF-8 and the cursor is at line 6, column 4.



The automation script identifies the search input field using the `By.name("search")` locator and injects the 'Laptop' search query via the `sendKeys()` method. Upon triggering the `submit()` command, the Selenium WebDriver successfully interacts with the web application to fetch and display the relevant product results. The browser instance confirms that the search functionality is operating as expected under automated control, providing a successful end-to-end test of the search feature.

Step 5: Verification & Cleanup

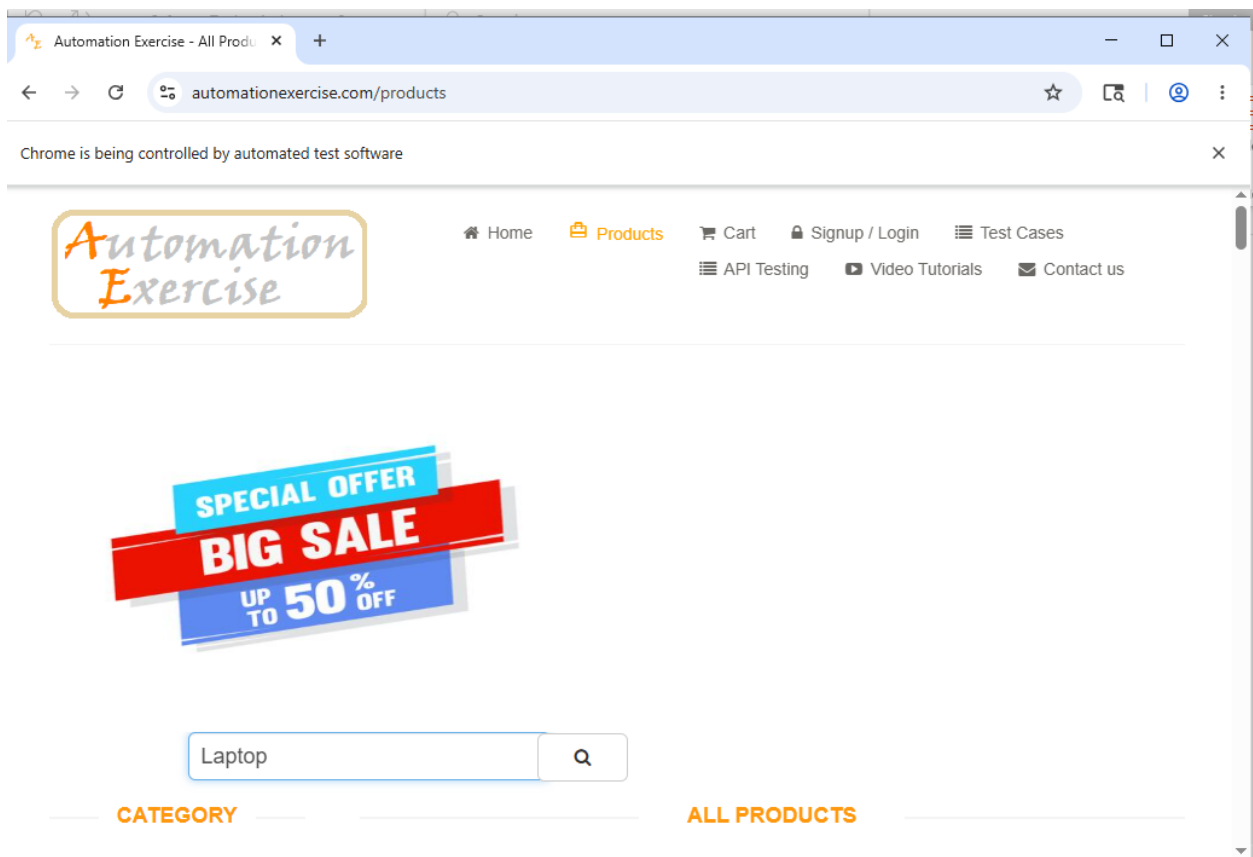


The screenshot shows an IDE with a project named 'SeleniumAssignment'. The file explorer on the left shows the project structure, including 'src/main/java/org/example' and 'target'. The main editor displays the 'EcommerceSearchTest.java' file with the following code:

```
8 public class EcommerceSearchTest {
9     public static void main(String[] args) {
10
11         WebDriver driver = new ChromeDriver();
12
13         driver.get("https://automationexercise.com/products");
14         driver.manage().window().maximize();
15         driver.findElement(By.name("search")).sendKeys(KeysToSend.Laptop, Keys.ENTER);
16         String title = driver.getTitle();
17         System.out.println("Page Title: " + title);
18         if (title.contains("All Products")) {
19             System.out.println("Test Status: PASSED");
20         } else {
21             System.out.println("Test Status: FAILED");
22         }
23     }
24     driver.quit();
25 }
26 }
```

The Run console at the bottom shows the following output:

```
May 12, 2026 11:21:10 PM org.openqa.selenium.chromium.ChromiumDriver.lambda$new$5
WARNING: Unable to find version of CDP to use for 148.0.7778.97. You may need to include a dependency on a specific version of the CDP using something sim
Page Title: Automation Exercise - All Products
Test Status: PASSED
Process finished with exit code 0
```



The final phase of the script involves executing the `driver.quit()` method. This command is responsible for closing all browser windows associated with the WebDriver session and safely terminating the driver process. Properly closing the session ensures that system resources are freed and prevents unnecessary background processes from persisting after the test execution is complete.

Conclusion

The automation script was successfully developed and executed, demonstrating the core capabilities of Selenium WebDriver. Through this task, proficiency was gained in environment configuration, Maven project management, and automated UI interaction. This exercise confirms the effectiveness of Selenium in streamlining repetitive testing processes.